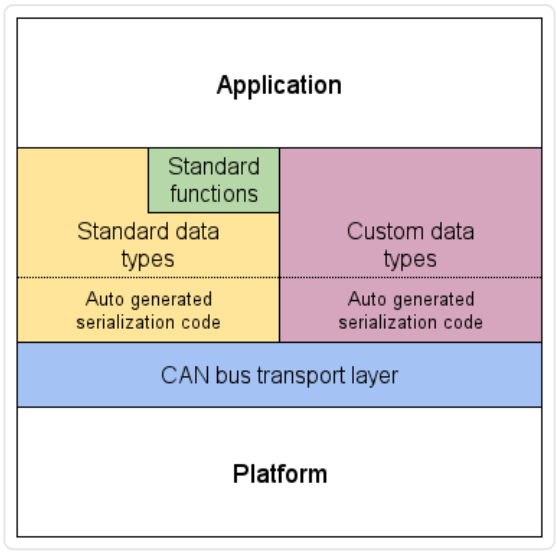
Basic concepts

DroneCAN is a lightweight protocol designed to provide a highly reliable communication method for aerospace and robotic applications via the CAN bus. The DroneCAN network is a decentralized peer network, where each peer (node) has a unique numeric identifier - *node ID*. The nodes of the DroneCAN network can communicate using any of the following communication methods:



- Message broadcasting - The primary method of data exchange with publish/subscribe semantics.
- Service invocation - The communication method for peer-to-peer request/response interactions.

For each type of communication, a predefined set of data structures is used, where each data structure has a unique identifier - the *data type ID* (DTID). Some data structures are standard and defined by the protocol specification; others may be specific to a particular application or vendor.

Since every published message type has its own unique data type ID, and each node of the network has its own unique node ID, a pair of data type ID and node ID can be used to support redundant nodes with identical functionality inside the same network.

Message and service data structures are defined using the *data structure description language* (DSDL). The DSDL description is used to generate the serialization/deserialization code for a given data structure in each target programming language. The DSDL approach allows compilers to determine the data structure size statically, thus helping to optimize the protocol implementations in terms of memory consumption and performance. This feature is important for deeply embedded systems, where the memory footprint is critical and dynamic memory allocation may not be acceptable.

On top of the standard data types, DroneCAN defines a set of standard high-level functions including: node health monitoring, network discovery, time synchronization, firmware update, and more. For more information see the part of the specification dedicated to the standard data types and application level functions.

Serialized message and service data structures are exchanged by means of the CAN bus transport layer, which implements automatic decomposition of long transfers into several CAN frames, allowing nodes to exchange data structures of arbitrary size.

Message broadcasting

Message broadcasting refers to the transmission of a serialized data structure over the CAN bus to other nodes. This is the primary DroneCAN data exchange mechanism. Typical use cases may include transfer of the following kinds of data (either cyclically or on an adhoc basis): sensor measurements, actuator commands, or equipment status information.

A broadcast message includes the following information:

Specification

1. Introduction
2. Basic concepts
3. Data structure description language
4.1 CAN bus transport layer
4.2 UDP bus transport layer
4.3 MAVLink bus transport layer
4.4 CANFD bus transport layer
5. Application level conventions
6. Application level functions
7. List of standard data types
8. Hardware design recommendations
dsdl
  ardupilot
  uavcan

Contents

Basic concepts
  Message broadcasting
    Anonymous message broadcasting
    Service invocation

License

This work is licensed under a Creative Commons Attribution 4.0 International License.
Much of the content of this site is based upon prior work by Pavel Kirienko and the UAVCAN Development Team.

Data type ID	Numerical identifier that indicates how the data structure should be interpreted
Source node ID	The node ID of the transmitting node
Transfer ID	A small overflowing integer that increments with every transfer of this type of message from a given node

Anonymous message broadcasting

Nodes that don't have a unique node ID can publish *anonymous messages*. An anonymous message is different from a regular message in that it doesn't contain source node ID. This kind of data exchange is useful during initial configuration of the node, particularly during dynamic node ID allocation procedure.

Service invocation

Service invocation is a two-step data exchange between exactly two nodes: a client and a server. The steps are:

1. The client sends a service request to the server.
2. The server takes appropriate actions and sends a response to the client.

Typical use cases for this type of communication include: node configuration parameter update, firmware update, adhoc action request, file transfer, and other service tasks.

Both service requests and service responses include the following data:

Field	Content
Payload	The serialized data structure
Data type ID	Numerical identifier that indicates how the data structure should be interpreted
Client node ID	Source node ID during request transfer, destination node ID during response transfer
Server node ID	Destination node ID during request transfer, source node ID during response transfer
Transfer ID	A small overflowing integer that increments with every call to this service from a given node

Both request and response contain exactly the same values for all fields except payload, where the content is application defined. Clients can match the response with a corresponding request using the following fields: data type ID, client node ID, server node ID, and transfer ID.